IluminaBH – Especificação de Publicação e Roadmap

Sistema Inteligente de Análise da Iluminação Pública de Belo Horizonte
Time de Engenharia de IA | PUC Minas | Integrantes: Guilherme Martini Brina Ferreira e Pedro Rodrigues
Repositório: https://github.com/Gmbferreira/Ilumina-bh-IA | Documento de 08/11/2026 | Entrega final: 06/12/2026

1. Estratégia de publicação

O IluminaBH será publicado como uma **aplicação web de execução local**, distribuída por um repositório público no GitHub. Essa escolha segue a arquitetura já definida e os requisitos RNF01 (operação local, sem dependência de serviços externos), RNF02 (sem serviços pagos ou nuvem obrigatória) e RNF09 (dados processados na máquina, sem envio a terceiros). O sistema roda em uma única máquina, sem GPU, com o servidor FastAPI acessível apenas em 127.0.0.1.

A publicação consiste em três artefatos, todos no GitHub e sem custo:

- **Código-fonte e documentação:** módulos separados de ingestão, tratamento, armazenamento, API, pipeline de ML e interface (RNF10), com README de instalação, execução, arquitetura, metodologia de IA, testes e limitações (RNF12).
- **Release versionada (tag v1.0, 06/12/2026):** pacote com o código, o arquivo requirements.txt com versões fixas das bibliotecas e o banco SQLite já populado e as execuções de ML registradas, para permitir a demonstração sem reprocessar tudo.
- **Dados brutos das bases da PBH:** baixados do Portal de Dados Abertos e guardados sem alteração em pasta própria. Conforme o volume e a licença do portal, o repositório pode trazer apenas um script de download e uma amostra controlada para testes.

2. Tecnologias e plataformas

Função	Tecnologia / plataforma	Custo	Observação
Linguagem e dados	Python 3, pandas, GeoPandas, Shapely	Gratuito	Ingestão, validação e conversão de CRS para EPSG:4326
Machine Learning	scikit-learn (K-Means, métricas)	Gratuito	CPU, treino em minutos, semente fixa e registrada
Banco de dados	SQLite (arquivo local)	Gratuito	PostGIS apenas como evolução (Seção 4)
Backend	FastAPI + Uvicorn em 127.0.0.1	Gratuito	Endpoints de dados, indicadores, qualidade e perfis
Frontend	HTML, CSS, JavaScript, Leaflet, Plotly	Gratuito	Bibliotecas empacotadas localmente; mapa-base OSM opcional
Testes e CI	pytest + GitHub Actions	Gratuito	Testes automatizados a cada commit e pull request (RNF11)
Versionamento	Git / GitHub	Gratuito	Código, documentação, Issues e Releases

3. Fluxo de publicação e instalação

Integração contínua. Um workflow do GitHub Actions instala as dependências, executa o pytest (regras de validação, indicadores e reprodutibilidade do agrupamento) e registra o resultado. A Release só é criada com a suíte de testes aprovada.

Instalação pelo usuário, em cinco passos documentados: (1) clonar o repositório ou baixar a Release; (2) criar o ambiente virtual e instalar o requirements.txt; (3) executar o comando de carga, que importa, valida e grava os dados; (4) executar o pipeline de ML por linha de comando; (5) iniciar o servidor e abrir o endereço local no navegador. Um integrante que não participou da implementação executará esses passos em outra máquina para validar RNF05 e RNF12.

Por que não usar nuvem na primeira entrega. As plataformas de IA de AWS (SageMaker), Azure (Azure Machine Learning) e Google Cloud (Vertex AI) foram avaliadas e descartadas por agora: o problema é um agrupamento leve em CPU, que não exige treino gerenciado, e a hospedagem em nuvem exigiria enviar os dados para serviços externos, conflitando com RNF01, RNF02 e RNF09 e criando risco de custo. Elas permanecem como alternativa futura, caso o escopo mude.

4. Roadmap da aplicação

Não há outra entrega além da final, portanto o roadmap é apresentado como uma única entrega, com início na data deste documento (08/11/2026) e fim em 06/12/2026, seguida de trabalhos futuros. As dez fases são sequenciais, e uma fase só é concluída quando a anterior está funcionando.

Período	Fases (etapas da Figura 1)	Resultado esperado
08/11 a 14/11	1 Carregar dados (1, 2); 2 Limpar e validar (3); 3 Banco de dados (4)	Bases importadas, relatório de qualidade e dados tratados no SQLite com contagem lidos = aceitos + rejeitados
15/11 a 21/11	4 Mapa (5, 10); 5 Filtros e relatórios (5, 10)	API FastAPI, mapa com unidades e trechos, filtros, indicadores e exportação CSV
22/11 a 28/11	6 Engenharia de atributos (6); 7 Agrupamento e avaliação (7, 8); 8 Integrar perfis ao mapa (9, 10)	K-Means com k de 2 a 10, métricas registradas, escolha documentada de k e perfis coloridos no mapa
29/11 a 06/12	9 Testes (todas); 10 Documentação (todas); publicação da Release v1.0	Testes executados e registrados, tempos medidos (RNF03), README validado e Release no GitHub em 06/12/2026

Marco de 06/12/2026 (entrega final): fluxo completo demonstrável – Importação, Validação, Limpeza, Armazenamento, Visualização, Filtragem, Indicadores, Preparação para IA, Agrupamento, Avaliação, Interpretação e Registro da execução.

5. Trabalhos futuros

A evolução prevista após a entrega final é a migração controlada para **PostgreSQL/PostGIS**. Ela será considerada se as medições de desempenho (RNF03) mostrarem que as consultas geográficas sobre o SQLite, em especial os filtros por área visível do mapa e o cálculo de densidade local, não atendem a uma demonstração interativa. A migração manteria a mesma API e as mesmas tabelas lógicas, com índices espaciais, e seria validada pelos testes automatizados já existentes, preservando a compatibilidade com a Lista de Requisitos.

Qualquer alteração será feita de forma controlada e documentada, sem comprometer o prazo de 06/12/2026 e mantendo as restrições de escopo: o sistema não detecta defeitos, não prevê falhas e não classifica a qualidade da iluminação.